

BTreeFy

Blackboard–Runner Notification Design Report

Victor Oliveira

February 2026

Contents

1	Problem Statement	3
1.1	Current Architecture	3
2	Approach A — Callback (Runner Passes Blackboard)	4
2.1	Dependency Flow	4
2.2	Key Changes	4
3	Approach B — Event Bus / Mediator	6
3.1	Dependency Flow	6
3.2	Key Changes	6
4	Approach C — Callback (Tree Holds Blackboard)	8
4.1	Dependency Flow	8
4.2	Key Changes	8
5	Comparison	9
5.1	Pros & Cons	9
5.2	Software Engineering Principles	9
5.3	Comparison with Industry-ready Behavior Tree Libraries	9
6	Recommendation: Approach A	11
6.1	Final Shape	11

1 Problem Statement

In `btf_blackboard.c`, the function `btf_blackboard_update_data` needs to notify the runner that the blackboard has changed:

```
btf_runner_notify_event(BTF_RUNNER_BLACKBOARD_EVT);
```

However, `btf_runner_notify_event` requires a `struct btf_runner*` parameter, and the blackboard has no reference to a runner object.

Adding a runner pointer directly to `struct btf_blackboard` would create a **cyclic dependency**:

Runner → Tree → Blackboard → **Runner x**

1.1 Current Architecture

The ownership chain is:

- A **runner** holds a pointer to a **tree** (`btf_tree_st`)
- A **tree** holds an array of **nodes** (`btf_node[]`)
- The **blackboard** is a standalone static object, defined via macros (`BTF_BLACKBOARD_DEFINE`), with no upward reference

The blackboard is architecturally **independent** — it does not belong to the tree or runner. This means there is no natural path from blackboard to runner.

Additionally, there is an inconsistency between runner implementations:

	Zephyr Runner	POSIX Runner
Notify signature	<code>btf_runner_notify_event(uint32_t evt)</code>	<code>btf_runner_notify_event(struct btf_runner *runner, uint32_t evt)</code>
Event mechanism	Global <code>K_EVENT</code>	Per-runner <code>pthread_cond_t</code>
Runner ref needed?	No (global state)	Yes

Table 1: Runner implementation inconsistency

2 Approach A — Callback (Runner Passes Blackboard)

The blackboard gets a **generic callback** (`on_update`) — a function pointer and an opaque `void*` context. The runner passes itself as the context when registering. The blackboard has zero knowledge of the runner.

2.1 Dependency Flow

```
Runner → (passes blackboard param & registers callback) → Blackboard
Blackboard → (calls generic void(*fn)(void*)) → Runner callback
```

2.2 Key Changes

2.2.1 `btft_blackboard.h` — New callback fields

```
typedef void (*btft_blackboard_notify_fn_t)(void *context);

struct btft_blackboard
{
    void *data;
    size_t data_size;
    btft_blackboard_notify_fn_t on_update; // NEW
    void *on_update_context; // NEW
};
```

2.2.2 `btft_blackboard.c` — Calls callback (no runner include)

```
// NOTE: #include "btft_tree_runner.h" is REMOVED

int32_t btft_blackboard_update_data(struct btft_blackboard *blackboard,
                                   size_t offset, void *data, size_t size)
{
    memcpy((uint8_t *) blackboard->data + offset, data, size);
    if (blackboard->on_update != NULL)
        blackboard->on_update(blackboard->on_update_context);
    return 0;
}
```

2.2.3 `btft_tree_runner_posix.c` — Bridge callback and registration

```
static void blackboard_change_callback(void *context)
{
    struct btft_runner *runner = (struct btft_runner *) context;
    btft_runner_notify_event(runner, BTF_RUNNER_BLACKBOARD_EVT);
}
```

```
int32_t btf_runner_init(struct btf_runner *runner, btf_tree_st *tree,
                       struct btf_blackboard *blackboard, // NEW param
                       struct btf_runner_config *config)
{
    // ...
    if (blackboard != NULL)
        btf_blackboard_set_notify(blackboard,
                                   blackboard_change_callback, runner);
    // ...
}
```

Tip

The `BTF_RUNNER_BLACKBOARD_EVT` constant is only referenced **inside the runner's bridge callback**, never in the blackboard code. This keeps the blackboard fully agnostic.

3 Approach B — Event Bus / Mediator

A standalone **event bus module** acts as a central mediator. The blackboard publishes events to the bus; the runner subscribes to the bus. Neither module knows the other exists.

3.1 Dependency Flow

```

Blackboard → Event Bus ← Runner
Event Bus → (dispatches to) → Runner callback
  
```

3.2 Key Changes

3.2.1 `btf_event_bus.h` — New module

```

#define BTF_EVENT_BUS_MAX_SUBSCRIBERS 4

typedef void (*btf_event_bus_handler_t)(uint32_t event, void *context);

void    btf_event_bus_init(void);
int32_t btf_event_bus_subscribe(uint32_t event_mask,
                                btf_event_bus_handler_t handler,
                                void *context);
void    btf_event_bus_publish(uint32_t event);
  
```

3.2.2 `btf_blackboard.c` — Publishes via bus

```

#include "btf_event_bus.h" // instead of btf_tree_runner.h

int32_t btf_blackboard_update_data(...)
{
    memcpy(...);
    btf_event_bus_publish(BTF_RUNNER_BLACKBOARD_EVT);
    return 0;
}
  
```

3.2.3 `btf_tree_runner_posix.c` — Subscribes via bus

```

static void runner_event_bus_handler(uint32_t event, void *context)
{
    struct btf_runner *runner = (struct btf_runner *) context;
    btf_runner_notify_event(runner, event);
}

int32_t btf_runner_init(...)
{
    // ...
    btf_event_bus_subscribe(BTF_RUNNER_BLACKBOARD_EVT,
  
```

```
runner_event_bus_handler, runner);  
// ...  
}
```

Note

The event bus uses a static array of subscribers with a compile-time maximum (BTF_EVENT_BUS_MAX_SUBSCRIBERS = 4), suitable for embedded targets. No dynamic allocation.

4 Approach C — Callback (Tree Holds Blackboard)

The `tree` (`btf_tree_st`) holds a `struct btf_blackboard*`. The runner accesses it via `tree->blackboard` during init and registers a callback. The `btf_runner_init` signature stays unchanged.

4.1 Dependency Flow

```
Runner → Tree → (holds NEW) → Blackboard
Runner → (registers callback via tree->blackboard) → Blackboard
```

4.2 Key Changes

4.2.1 `btreefy_objs.h` — Tree holds blackboard pointer

```
struct btf_blackboard; // forward declaration (no include)

typedef struct
{
    struct btf_node      *nodes;
    uint32_t             size;
    uintptr_t            running_node_index;
    struct btf_blackboard *blackboard; // NEW
} btf_tree_st;
```

4.2.2 `btf_tree_runner_posix.c` — Reads `tree->blackboard`

```
int32_t btf_runner_init(struct btf_runner *runner, btf_tree_st *tree,
                        struct btf_runner_config *config) // UNCHANGED
{
    // ...
    if (tree->blackboard != NULL)
        btf_blackboard_set_notify(tree->blackboard,
                                   blackboard_change_callback, runner);
    // ...
}
```

4.2.3 Application init

```
btf_init(&tree, nodes, tree_size);
tree.blackboard = &btf_blackboard_obj_app_blackboard; // manual
btf_runner_init(&runner, &tree, &config);
```

Warning

The tree becomes a *carrier* for the blackboard reference, even though the tree itself never uses it — only the runner does. `tree.blackboard` must be set **before** `btf_runner_init` is called.

5 Comparison

5.1 Pros & Cons

Criterion	A — Callback (param)	B — Event Bus	C — Callback (via tree)
No cyclic dependency	✓	✓	✓
Blackboard decoupled	✓	✓	✓
No new modules	✓	✗	✓
No <code>btf_runner_init</code> change	✗	✓	✓
No change to <code>btf_tree_st</code>	✓	✓	✗
No unnecessary stored refs	✓	✓	⚠
Multiple observers	✗	✓	✗
Memory overhead	~16 B	~64 B	~16 B + 1 ptr
Init ordering sensitivity	Low	Low	⚠ High

Table 2: Feature comparison of the three approaches

5.2 Software Engineering Principles

Principle	A	B	C
Single Responsibility	✓	✓	⚠
Dependency Inversion	✓	✓	✓
Low Coupling	✓	✓	⚠
High Cohesion	✓	⚠	⚠
KISS	✓	✗	✓
YAGNI	✓	⚠	✓

Table 3: Evaluation against software engineering principles

5.3 Comparison with Industry-ready Behavior Tree Libraries

Research into established, industry-standard Behavior Tree implementations confirms that the **Callback/Observer pattern (Approach A)** aligns perfectly with best practices for building dynamic, reactive behavior trees.

The primary necessity for this pattern is achieving **Reactive Execution**. A naive behavior tree requires the runner to constantly poll (`tick()`) the tree to detect changes in the environment. By allowing the blackboard to notify the runner of events, the runner can sleep (e.g., via `pthread_cond_wait`), saving valuable CPU cycles. This is crucial for resource-constrained embedded systems and is handled similarly in major frameworks:

Unreal Engine Behavior Trees: Unreal Engine implements exactly this design via “Blackboard Observers.” When the tree executes, decorators subscribe to specific blackboard keys. Upon modification, the blackboard emits a notification, alerting the engine to re-evaluate branches, triggering “Observer Aborts”, all without the blackboard needing specific knowledge of the nodes.

BehaviorTree.CPP: This popular robotics library fundamentally relies on polling via Port communication during the tick cycle. Consequently, the community frequently discusses the need for event-driven reactive updates like reacting to asynchronous ROS messages. Developers often have to build custom wrappers or decorators to simulate the exact callback mechanism proposed here. Implementing the observer natively inside `btf_blackboard.c` is actually a step ahead of standard `BehaviorTree.CPP` polling mechanisms.

NodeCanvas and Behavior Designer: These Unity tools leverage the C-sharp event system under the hood. Their Shared Variables, also known as Blackboards, fire `delegate` callbacks upon modification. The tree engine subscribes to these delegates during initialization—directly mirroring the top-down dependency flow defined in Approach A.

In summary, implementing **Approach A** with a callback registered via dependency injection in `btf_runner_init` is technically consistent with top-tier reactive BT engines like Unreal’s, while remaining perfectly suited for the memory and scale constraints of embedded C programming.

6 Recommendation: Approach A

Approach A (Callback — runner passes blackboard) is the best fit for BTreeFy:

1. **Cleanest SRP** — the tree stays purely a tree; the blackboard is purely a data store with a notification hook; the runner is the only module aware of both.
2. **Explicit dependency** — the blackboard parameter in `btf_runner_init` makes the dependency *visible at the API level* rather than hidden inside a struct field or a global module.
3. **KISS + YAGNI** — no new modules, no conceptual stretching of the tree's purpose, no global state. It solves exactly the problem with minimal additions.
4. **The signature change is a feature** — it documents the runner's dependency on the blackboard for anyone reading the API.

Tip

Do not store `runner->blackboard`. Use the parameter locally in `btf_runner_init` to call `btf_blackboard_set_notify`, then discard it. The runner struct stays unchanged.

6.1 Final Shape

```
int32_t btf_runner_init(struct btf_runner *runner,
                      btf_tree_st *tree,
                      struct btf_blackboard *blackboard,
                      struct btf_runner_config *config);

// Inside btf_runner_init:
if (blackboard != NULL)
{
    btf_blackboard_set_notify(blackboard,
                             blackboard_change_callback,
                             runner);
}
// No runner->blackboard = blackboard; (not needed)
```